Automated AI News Aggregator & Digest System

Academic Project Presentation Guide & Architecture Specification

1. Executive Summary & Problem Statement

The Problem: Artificial Intelligence research advances rapidly across tech blogs (OpenAI, Hugging Face, Google AI) and video channels. For students, researchers, and professionals, manually checking dozens of websites daily leads to severe **information overload**.

The Solution: An automated software pipeline that:
• **Aggregates:** Fetches RSS/Atom XML streams and YouTube channel feeds concurrently.
• **Deduplicates:** Stores records in a relational DB with unique URL constraints to prevent repeat processing.
• **Summarizes (GenAI):** Employs **Google Gemini 3.6 Flash LLM** to extract 3 bullet points, a bottom-line impact statement, and a topic classification tag.
• **Dispatches:** Compiles responsive HTML newsletters and emails them via encrypted SMTP TLS.

2. System Architecture & Execution Flow

Phase	Module / Layer	Core Responsibility
1. Ingestion	app/ingest/ (blogs.py, youtube.py)	Parses RSS/XML feeds and cleans HTML markup using BeautifulSoup4.
2. Persistence	app/database.py & models.py	SQLAlchemy ORM. Deduplicates records via unique URL index constraint.
3. GenAI Summarization	app/llm/ (agent.py, prompts.py)	Calls Gemini 3.6 Flash API using structured JSON schema. Applies rate throttling.
4. Notification	app/notify/ (email.py)	Compiles responsive HTML cards & sends to distribution list via SMTP TLS.
5. Controller	app/main.py & render.yaml	Orchestrates pipeline execution on a daily automated cron schedule.

3. How Generative AI (GenAI) is Implemented

• **Model Selection (Gemini 3.6 Flash):** Configured with Google's latest multimodal LLM model for high-throughput comprehension and low latency.

• **System Persona Prompting:** Establishes an 'Expert AI & Tech Journalist' system persona (app/llm/prompts.py) to guide summarization quality.

• **Guaranteed JSON Output (response_mime_type='application/json'):** Instructs Gemini to return strict JSON data matching a pre-defined schema, bypassing fragile regex parsing.

• **Rate-Limiting Throttling:** Implements intelligent sleeping delays (time.sleep(12)) to comply with free-tier 5 requests/min API quotas.

• **Fault-Tolerant Fallbacks:** Degrades gracefully to cleaned text snippets if LLM quotas or network timeouts occur.

4. Complete File-by-File Technical Role Summary

File / Path	Component Type	Key Technical Function
app/config.py	Configuration	Pydantic BaseSettings loading .env, parsing feed/email lists.
app/database.py	Database Layer	SQLAlchemy engine, pool_pre_ping=True, init_db() helper.
app/models.py	ORM Schema	NewsItem table schema with indexed unique url constraint.
app/ingest/blogs.py	Ingestion Engine	RSS/Atom parser & BeautifulSoup HTML text cleaner.

app/ingest/youtube.py	Ingestion Engine	YouTube XML channel feed scraper for video releases.
app/llm/prompts.py	LLM Prompts	System instructions & JSON schema template definitions.
app/llm/agent.py	LLM Client	google-genai Client wrapper & JSON response parser.
app/notify/email.py	Notifier	Responsive HTML newsletter compiler & multi-recipient SMTP dispatcher.
app/main.py	Orchestrator	End-to-end controller (Ingest -> Deduplicate -> Summarize -> Email).
render.yaml	IaC Infrastructure	Render Cron Job blueprint for automated daily cloud execution.